CPU Scheduling Simulator

Implemented by : Aashish Kumar

This project is a simple terminal-based simulator written in C to understand how different CPU scheduling algorithms behave in a multicore environment.

It currently supports:

- Lottery Scheduling
- Rate Monotonic Scheduling (RMS)

The simulator runs step-by-step (tick-wise), shows which task is running on each core, and calculates performance metrics like turnaround and waiting time.

Algorithms

1. Lottery Scheduling

Lottery scheduling works on a randomized approach.

- Every task is given a number of tickets
- At each step, one ticket is randomly picked
- The task owning that ticket gets CPU time
- Tasks with more tickets have a higher chance of being selected
- A time quantum of 4 is used before switching

2. Rate Monotonic Scheduling (RMS)

RMS is a priority-based preemptive algorithm mainly used in real-time systems.

- Each task has a fixed period
- Priority depends on the period: smaller period → higher priority
- If a higher priority task arrives, it immediately preempts the current one

Usage

The tasks used in the simulation are predefined in the program. So, you don't need to enter process details manually.

When you run the program:

1. A menu is displayed: Run Lottery Scheduling, Run Rate Monotonic Scheduling (RMS), or Exit
2. The selected scheduling algorithm is executed and tasks are initialized internally before every run
3. The simulator outputs tick-by-tick execution across CPU cores, task allocation per core, and final performance statistics

Sample Test Runs

1: Lottery Scheduling

Execution Timeline

```
--- Commencing Lottery Scheduling (Quantum = 4) ---
Tick    Core 0  Core 1
----------------------------------
00      [T1]    [--]
01      [T1]    [T2]
02      [T1]    [T2]
03      [T1]    [T2]
04      [T3]    [T2]
05      [T3]    [T2]
06      [T3]    [T2]
07      [T4]    [T2]
08      [T4]    [T2]
09      [T4]    [T1]
10      [T4]    [T1]
11      [T5]    [T1]
12      [T5]    [T1]
13      [T6]    [T7]
14      [T6]    [T7]
...     ...     ...
135     [T12]   [--]
136     [T12]   [--]
```

Performance Statistics

```
TID  Arrival  Burst  Start  Finish  Turnaround  Waiting
---  -------  -----  -----  ------  ----------  -------
T1       0      8      0      13        13          5
T2       1     25      1      52        51         26
T3       2      3      4       7         5          2
T4       5     12      7      26        21          9
T5       8      2     11      13         5          3
T6      10     40     13     112       102         62
T7      12      5     13      18         6          1
T8      15      6     30      39        24         18
```

```
T9      20     15     21      51        31         16
T10     22      4     25      29         7          3
T11     25      8     29      37        12          4
T12     30     50     43     137       107         57
T13     35      2     38      40         5          3
T14     40      9     47      68        28         19
T15     45      3     56      59        14         11
T16     50     18     51      70        20          2
T17     55      5     59      71        16         11
T18     60      7     63      74        14          7
T19     65     22     67      96        31          9
T20     70      4     71      75         5          1
```

Averages

```
Mean Turnaround Time : 25.85
Mean Waiting Time    : 13.45
```

Observations

- The scheduling order is not fixed and may change in different runs because the algorithm is randomized
- Tasks with higher ticket counts tend to get CPU time more frequently
- Some tasks (like T2, T6, T12) appear multiple times due to higher probability
- Short tasks finish early if selected quickly, but may also get delayed due to randomness
- Long-running tasks (such as T6 and T12) occupy CPU for extended periods once selected
- Compared to RMS, waiting and turnaround times are generally higher due to lack of strict priority

2: Rate Monotonic Scheduling (RMS)

Execution Timeline

```
--- Commencing Rate Monotonic Scheduling (RMS) ---
Priority = shortest period first (preemptive)
Tick    Core 0  Core 1
---------------------------------
00      [T1]    [--]
01      [T1]    [T2]
02      [T1]    [T3]
03      [T1]    [T3]
04      [T1]    [T3]
05      [T1]    [T4]
06      [T1]    [T4]
07      [T1]    [T4]
08      [T5]    [T4]
09      [T5]    [T4]
10      [T2]    [T4]
11      [T2]    [T4]
12      [T7]    [T4]
```

```
13      [T7]    [T4]
14      [T7]    [T4]
15      [T7]    [T8]
16      [T7]    [T8]
...     ...     ...
138     [T12]   [--]
139     [T12]   [--]
```

Performance Statistics

TID	Arrival	Burst	Start	Finish	Turnaround	Waiting
T1	0	8	0	8	8	0
T2	1	25	1	69	68	43
T3	2	3	2	5	3	0
T4	5	12	5	19	14	2
T5	8	2	8	10	2	0
T6	10	40	37	109	99	59
T7	12	5	12	17	5	0
T8	15	6	15	21	6	0
T9	20	15	20	36	16	1
T10	22	4	22	26	4	0
T11	25	8	25	33	8	0
T12	30	50	90	140	110	60
T13	35	2	35	37	2	0
T14	40	9	40	49	9	0
T15	45	3	45	48	3	0
T16	50	18	50	68	18	0
T17	55	5	55	60	5	0
T18	60	7	60	67	7	0
T19	65	22	68	90	25	3
T20	70	4	70	74	4	0

Averages

```
Mean Turnaround Time : 20.80
Mean Waiting Time    :  8.40
```

Observations

- Tasks with shorter periods get higher priority
- Preemption ensures faster response for high-priority tasks
- Overall waiting and turnaround times are lower compared to Lottery Scheduling

Limitations

- Lottery scheduling output is non-deterministic (random ticket selection)
- RMS does not simulate periodic task re-release (tasks run only once)
- No support for I/O or blocked states (all tasks are CPU-bound)

Implemented by : Abhinav Kumar

This project is a terminal-based CPU scheduling simulator written in **C** that demonstrates how different scheduling algorithms behave in a **multiprocessor environment**.

It supports:

- **Round Robin Scheduling (RR)**
- **Multilevel Feedback Queue (MLFQ)**

The simulator prints a **tick-by-tick execution timeline (Gantt Chart)** and calculates performance metrics such as turnaround time, waiting time, and response time.

Project Structure

```
main.c       - Handles user input and controls execution
algorithms.c - Implements scheduling logic and timeline display
algorithms.h - Contains data structures and function prototypes
Makefile     - Used to build and run the project
README.md    - Project documentation
```

Algorithms

1. Round Robin (RR)

- Uses **time quantum = 4**
- Processes are scheduled in **cyclic order**
- Each process gets CPU for a fixed time slice
- If not completed, it goes back to the ready queue

2. Multilevel Feedback Queue (MLFQ)

RMS is a priority-based preemptive algorithm mainly used in real-time systems.

- Uses **multiple priority queues**
- Higher priority queues execute first
- If a process uses full quantum → it is **demoted**
- Lower queue behaves like **FCFS**

Queue Configuration:

- Q0 → Quantum = 4
- Q1 → Quantum = 8
- Q2 → FCFS (no quantum limit)

Usage

The tasks used in the simulation are **predefined** in the program.
So, you don't need to enter process details manually.

When you run the program:

1. Scheduling algorithm is executed

2. System simulates execution across multiple cores

3. Output includes::

    - Tick-by-tick Gantt chart
    - CPU allocation per core
    - Final performance statistics (Turnaround Time & Waiting Time)

Sample Test Runs

1: Round Robin Scheduling

Execution Timeline

```
--- Commencing Round Robin ----

Tick    Core 0  Core 1
---------------------------------
00      [P1]    [--]
01      [P1]    [P2]
02      [P1]    [P2]
03      [P1]    [P2]
04      [P1]    [P2]
05      [P1]    [P2]
06      [P1]    [P2]
07      [P1]    [P2]
08      [P3]    [P2]
09      [P3]    [P2]
10      [P3]    [P2]
11      [P4]    [P2]
12      [P4]    [P2]
13      [P4]    [P2]
14      [P4]    [P2]
15      [P4]    [P2]
16      [P4]    [P2]
...
127     [P19]   [--]
128     [P19]   [--]
```

Performance Statistics

```
TID     Arrival Burst   Start   Finish  Turnaround   Waiting
---     ------- -----   -----   ------  ----------   -------
P1          0      8      8       8       0            0
P2          1      25     26      25      0            0
P3          2      3      11      9       6            6
P4          5      12     23      18      6            6
P5          8      2      25      17      15           15
P6          10     40     65      55      15           15
P7          12     5      31      19      14           14
P8          15     6      37      22      16           16
P9          20     15     52      32      17           17
P10         22     4      56      34      30           30
P11         25     8      64      39      31           31
P12         30     50     114     84      34           34
P13         35     2      67      32      30           30
P14         40     9      76      36      27           27
P15         45     3      79      34      31           31
P16         50     18     97      47      29           29
P17         55     5      102     47      42           42
P18         60     7      109     49      42           42
P19         65     22     131     66      44           44
P20         70     4      118     48      44           44
```

Averages

```
Avg Turnaround Time : 36.05
Avg Waiting Time    : 23.65
Avg Response Time   : 23.65
```

2: MLFQ

Execution Timeline

```
--- Commencing Lottery Scheduling (Quantum = 4) ---
Tick    Core 0  Core 1
---------------------------------
00      [P1]    [--]
01      [P1]    [P2]
02      [P1]    [P2]
03      [P1]    [P2]
04      [P3]    [P2]
05      [P3]    [P4]
06      [P3]    [P4]
07      [P1]    [P4]
08      [P1]    [P4]
09      [P1]    [P5]
10      [P1]    [P5]
11      [P6]    [P2]
12      [P6]    [P2]
```

```
13      [P6]    [P2]
14      [P6]    [P2]
...
118     [P19]   [P12]
119     [P19]   [P12]
```

Performance Statistics

TID	Arrival	Burst	Start	Finish	Turnaround	Waiting
P1	0	8	11	11	3	0
P2	1	25	87	86	61	0
P3	2	3	7	5	2	2
P4	5	12	27	22	10	0
P5	8	2	11	3	1	1
P6	10	40	105	95	55	1
P7	12	5	38	26	21	3
P8	15	6	40	25	19	4
P9	20	15	90	70	55	3
P10	22	4	31	9	5	5
P11	25	8	48	23	15	2
P12	30	50	128	98	48	1
P13	35	2	37	2	0	0
P14	40	9	59	19	10	0
P15	45	3	50	5	2	2
P16	50	18	111	61	43	0
P17	55	5	65	10	5	1
P18	60	7	70	10	3	0
P19	65	22	121	56	34	0
P20	70	4	74	4	0	0

Averages

```
Avg Turnaround Time : 32.00
Avg Waiting Time    : 19.60
Avg Response Time   : 1.25
```

Observations

- Round Robin ensures fair CPU distribution among processes
- MLFQ improves performance by prioritizing shorter tasks
- Response time is minimized for high-priority processes
- Long processes are gradually pushed to lower priority queues

Limitations

- No support for I/O or blocked states (all processes are CPU-bound)
- MLFQ does not implement aging (possible starvation in lower queues)
- Fixed number of queues and quantum values

Implemented by Aditya

This project is a terminal-based simulator written in **C** to understand how different CPU scheduling algorithms behave in a multicore environment.

It currently supports:

- **Completely Fair Scheduler (CFS)**
- **Multilevel Feedback Queue (MLFQ)**

The simulator runs step-by-step (tick-wise), shows which task is running on each core, displays active IO wait states, and calculates performance metrics like turnaround and waiting time.

Project Structure

Combined project (CFS directory):

```
main.c          - Handles user input, menu, and task initialization
 algorithms.c    - Implements MLFQ and CFS scheduling logic with timeline
display
 algorithms.h    - Contains function prototypes
 types.h        - Data structures (Task, Core, ProcStatus)
 Makefile       - Used to build and run the project
```

Standalone MLFQ (MLFQ directory):

```
mlfq.c          - Self-contained MLFQ scheduler with main(), init, and
scheduling logic
 types.h        - MLFQ-specific Task and Core structures
 Makefile       - Build script
```

types.h — Task & Core structures

```c
#ifndef ADITYA_TYPES_H
#define ADITYA_TYPES_H

typedef enum { STATUS_READY, STATUS_RUN, STATUS_WAIT, STATUS_DONE } ProcStatus;

// Standard structure representing our system task
typedef struct {
    int task_id;
    int time_arrival;
    int time_burst;
    int time_rem;

    // MLFQ & IO specific tracking
    int io_freq;           // How often task triggers memory/IO
    int io_dur;            // How long task waits for IO
    int io_rem;            // Remaining ticks in current IO wait
    int cpu_burst_dur;     // Time spent on CPU since last IO wait
    int current_queue;     // 0: High priority, 1: Medium, 2: FCFS

    // CFS specific tracking
    double vruntime;       // Virtual runtime for CFS scheduling
    int weight;            // Weight based on niceness for CFS
    int nice_val;          // Nice value (-5 to 5 range in our sim)

    // Standard variables
    int prio_lvl;
    ProcStatus status;

    // Fields for stats
    int time_done;
    int time_start;
    int processor_id;
} Task;

// Represents simulated CPU compute cores
typedef struct {
    int core_id;
    int active_task_id; // mapped active task id, idle state mapped as -1
    int quantum_rem;    // track time slicing limits
} Core;

#endif
```

Figure: types.h — Task and Core data structures

algorithms.h — Function prototypes

```c
#ifndef ADITYA_ALGORITHMS_H
#define ADITYA_ALGORITHMS_H

#include "types.h"

void execute_mlfq_sched(Task task_list[], int total_tasks, int total_cores);
void execute_cfs_sched(Task task_list[], int total_tasks, int total_cores);

#endif
```

Figure: algorithms.h — Function prototypes

main.c — Entry point & task initialization

```c
#include <stdio.h>
#include <stdlib.h>
#include "algorithms.h"
#include "types.h"

#define MAX_TASKS 15
#define CORE_COUNT 2

// Set up initialization for all our core mock tasks
// This handles assigning predefined IO frequencies, durations, and nice values
void init_tasks(Task task_arr[], int total_tasks) {
    Task base_data[MAX_TASKS] = {
        // task_id, arrival, burst, time_rem, io_freq, io_dur, io_rem, cpu_burst_dur, current_queue,
        // vruntime, weight, nice_val, prio_lvl, status, time_done, time_start, processor_id

        // CPU bound tasks (0 IO freq) - these test pure scheduling behavior
        {  1,  0, 15, 15,  0,  0, 0, 0, 0,  0.0, 1024,  0, 0, STATUS_READY, 0, -1, -1 },
        // High IO frequency tasks - tests IO handling and queue promotion
        {  2,  1,  6,  6,  2,  3, 0, 0, 0,  0.0, 1024,  2, 0, STATUS_READY, 0, -1, -1 },
        {  3,  2, 10, 10,  4,  5, 0, 0, 0,  0.0, 1024, -2, 0, STATUS_READY, 0, -1, -1 },
        {  4,  4,  4,  4,  0,  0, 0, 0, 0,  0.0, 1024,  0, 0, STATUS_READY, 0, -1, -1 },
        {  5,  5, 20, 20,  8,  4, 0, 0, 0,  0.0, 1024, -5, 0, STATUS_READY, 0, -1, -1 },
        {  6,  8,  5,  5,  0,  0, 0, 0, 0,  0.0, 1024,  3, 0, STATUS_READY, 0, -1, -1 },
        {  7, 10, 12, 12,  3,  2, 0, 0, 0,  0.0, 1024, -1, 0, STATUS_READY, 0, -1, -1 },
        {  8, 12,  8,  8,  0,  0, 0, 0, 0,  0.0, 1024,  1, 0, STATUS_READY, 0, -1, -1 },
        {  9, 15, 18, 18,  5,  6, 0, 0, 0,  0.0, 1024,  5, 0, STATUS_READY, 0, -1, -1 },
        { 10, 18,  9,  9,  0,  0, 0, 0, 0,  0.0, 1024,  0, 0, STATUS_READY, 0, -1, -1 },
        { 11, 20,  3,  3,  1,  2, 0, 0, 0,  0.0, 1024, -3, 0, STATUS_READY, 0, -1, -1 },
        { 12, 22, 25, 25, 10,  5, 0, 0, 0,  0.0, 1024,  4, 0, STATUS_READY, 0, -1, -1 },
        { 13, 25,  7,  7,  3,  3, 0, 0, 0,  0.0, 1024, -4, 0, STATUS_READY, 0, -1, -1 },
        { 14, 28, 14, 14,  6,  4, 0, 0, 0,  0.0, 1024,  0, 0, STATUS_READY, 0, -1, -1 },
        { 15, 30,  5,  5,  0,  0, 0, 0, 0,  0.0, 1024,  0, 0, STATUS_READY, 0, -1, -1 }
    };
    for (int k = 0; k < total_tasks; k++) {
        task_arr[k] = base_data[k];
    }
}

int main() {
    int total_tasks = MAX_TASKS;
    int total_cores = CORE_COUNT;
    Task task_list[MAX_TASKS];

    int user_choice;

    printf("\n+++ Multiprocessor Scheduling Simulator (MLFQ & CFS) +++\n");
    printf("   Algorithms: MLFQ (Multilevel Feedback Queue) & CFS (Completely Fair Scheduler)\n");
    printf("   Number of Processing Cores: %d\n", total_cores);
    printf("   Total tasks initialized: %d\n\n", total_tasks);

    while (1) {
        init_tasks(task_list, total_tasks);

        printf("Available choices:\n");
        printf(" 1. Run MLFQ Scheduler\n");
        printf(" 2. Run CFS (Completely Fair Scheduler)\n");
        printf(" 3. Quit\n> ");

        scanf("%d", &user_choice);

        switch (user_choice) {
            case 1:
                execute_mlfq_sched(task_list, total_tasks, total_cores);
                break;
            case 2:
                execute_cfs_sched(task_list, total_tasks, total_cores);
                break;
            case 3:
                printf("\nExiting. See you later!\n");
                exit(0);
            default:
                printf("\nThat didn't work. Pick 1, 2, or 3.\n");
        }
    }

    return 0;
```

Figure: main.c — Menu-driven entry point & task initialization

Algorithms

1. Multilevel Feedback Queue (MLFQ)

MLFQ uses a **three-tiered priority queue** system to balance responsiveness for interactive tasks and throughput for CPU-bound tasks.

- All tasks start in **Q0 (highest priority)**
- If a task uses its full time quantum, it is **demoted** to the next lower queue
- Tasks completing IO are **promoted back to Q0** (IO boost)
- Lower queues receive CPU only when all higher queues are empty

Queue configuration:

Queue	Time Quantum	Behavior
`Q0`	2 ticks	Highest priority — interactive / IO-bound tasks
`Q1`	4 ticks	Medium priority — mixed workloads
`Q2`	8 ticks	Lowest priority — FCFS-like, CPU-bound tasks

Parameters used:

Parameter	Meaning
Burst Time	Total CPU execution time required
Arrival Time	When the task enters the system
IO Frequency	How often the task triggers an IO wait (in CPU ticks)
IO Duration	How long the task blocks during each IO wait

Scheduling logic (per tick):

- **Step 1:** Decrement IO timers; promote completed-IO tasks to Q0
- **Step 2:** Partition READY tasks into Q0/Q1/Q2 bucket arrays
- **Step 3:** Fill idle cores with tasks, strictly Q0 → Q1 → Q2
- **Step 4:** Display tick timeline
- **Step 5:** Execute 1 tick — if burst done → DONE; if IO triggered → WAIT; if quantum expired → demote

algorithms.c — execute_mlfq_sched()

```c
 *  MLFQ SCHEDULING ALGORITHM
 *  -------------------------
 *  Three-level feedback queue with IO handling:
 *    Q0 (quantum=2) -> Q1 (quantum=4) -> Q2 (quantum=8, FCFS-like)
 *  Tasks that complete IO burst are promoted back to Q0.
 * ================================================================== */

int get_quantum(int queue_level) {
    if (queue_level == 0) return QUANTUM_Q0;
    if (queue_level == 1) return QUANTUM_Q1;
    return QUANTUM_Q2; // FCFS behaves as big quantum or bounded by 8
}

void execute_mlfq_sched(Task task_arr[], int total_tasks, int core_count) {
    printf("\n--- Commencing MLFQ Scheduling ---\n");
    printf("Queues: Q0(Q=2) -> Q1(Q=4) -> Q2(Q=8)\n");
    printf("Tick\t");
    for (int k = 0; k < core_count; k++) {
        printf("Core %d\t", k);
    }
    printf("| External Wait States\n");
    printf("-------------------------------------------------------\n");

    Core proc_cores[core_count];
    for (int idx = 0; idx < core_count; idx++) {
        proc_cores[idx].core_id = idx;
        proc_cores[idx].active_task_id = -1;
        proc_cores[idx].quantum_rem  = 0;
    }

    int clock_tick  = 0;
    int done_tasks  = 0;

    while (done_tasks < total_tasks) {

        // Step 1: Manage IO processing / Waiting Tasks
        for (int t = 0; t < total_tasks; t++) {
            if (task_arr[t].status == STATUS_WAIT) {
                task_arr[t].io_rem--;
                if (task_arr[t].io_rem <= 0) {
                    // IO complete, promote to top queue Q0
                    task_arr[t].status = STATUS_READY;
                    task_arr[t].current_queue = 0;
                    task_arr[t].cpu_burst_dur = 0;
                }
            }
        }

        // Step 2: Extract current ready tasks into queue bucket arrays
        int q0[MAX_TASKS], q0_len = 0;
        int q1[MAX_TASKS], q1_len = 0;
        int q2[MAX_TASKS], q2_len = 0;

        for (int t = 0; t < total_tasks; t++) {
            if (task_arr[t].status == STATUS_READY && task_arr[t].time_arrival <= clock_tick) {
                if (task_arr[t].current_queue == 0) q0[q0_len++] = t;
                else if (task_arr[t].current_queue == 1) q1[q1_len++] = t;
                else q2[q2_len++] = t;
            }
        }

        // Step 3: Schedule READY tasks onto empty cores based on MLFQ strict priority
        for (int idx = 0; idx < core_count; idx++) {
            if (proc_cores[idx].active_task_id != -1) continue; // Skip busy cores
```

Figure: algorithms.c — MLFQ scheduling function

2. Completely Fair Scheduler (CFS)

CFS is inspired by the **Linux kernel CFS**. Rather than fixed priority queues, it tracks a **`virtual runtime`** (vruntime) for each task and always schedules the task that has received the least CPU time.

Core ideas:

- Every task tracks a **`vruntime`** — a weighted measure of CPU time consumed
- The task with the **smallest vruntime** is scheduled next
- High-weight (low-nice) tasks accumulate vruntime **slower**, so they naturally get more real CPU time
- Dynamic time slices `proportional to weight / total-weight × target-latency`
- **Sleeper fairness:** tasks returning from IO have their vruntime set to min(vruntime) among ready tasks
- **Preemption:** a ready task can preempt a running task if its vruntime is significantly lower (> 2.0 difference)

Weight table (from Linux `sched_prio_to_weight`, scaled down):

Nice Value	Weight
−5	3277
−4	2621
−3	2097
−2	1678
−1	1342
0 (default)	1024
+1	820
+2	655
+3	526
+4	423
+5	335

vruntime increment formula:

```
vruntime += (1024.0 / task_weight)  ×  time_executed
```

Dynamic timeslice formula:

```
slice = (task_weight / total_ready_weight) × target_latency
 target_latency = max(6, ready_count)        // scales with load
 minimum granularity = 1 tick
```

Scheduling logic (per tick):

- **Step 1:** Handle IO completion; apply sleeper fairness (vruntime = min_vruntime)
- **Step 2:** Check if running tasks exhausted their dynamic time slice
- **Step 3:** Assign idle cores the task with the smallest vruntime
- **Step 4:** Preemption check — preempt running tasks if a ready task's vruntime is significantly lower
- **Step 5:** Display CFS timeline with vruntime values
- **Step 6:** Execute 1 tick, update vruntime; handle completion / IO triggers

algorithms.c — execute_cfs_sched()

```
 * COMPLETELY FAIR SCHEDULER (CFS) ALGORITHM
 * =========================================
 * Inspired by the Linux CFS. The core idea:
 *   - Every task tracks a "virtual runtime" (vruntime).
 *   - The task with the SMALLEST vruntime is picked next.
 *   - vruntime increments are weighted by "nice" value:
 *       vruntime_increment = (1.0 / weight) * time_executed
 *     Tasks with higher weight (lower nice) accumulate vruntime slower,
 *     so they get more real CPU time.
 *   - Dynamic time slices are computed per-task proportional to weight.
 *   - IO handling: tasks that block for IO have their vruntime set to
 *     the minimum vruntime of all ready tasks upon return (sleeper fairness).
 *
 * Weight table (simplified):
 *   nice -5 => weight 3277
 *   nice -4 => weight 2621
 *   nice -3 => weight 2097
 *   nice -2 => weight 1678
 *   nice -1 => weight 1342
 *   nice  0 => weight 1024 (default)
 *   nice  1 => weight  820
 *   nice  2 => weight  655
 *   nice  3 => weight  526
 *   nice  4 => weight  423
 *   nice  5 => weight  335
 * ================================================================== */

// CFS weight lookup table (from linux sched_prio_to_weight, scaled down)
static const int cfs_weight_table[] = {
    3277,  // nice = -5
    2621,  // nice = -4
    2097,  // nice = -3
    1678,  // nice = -2
    1342,  // nice = -1
    1024,  // nice =  0  (default weight)
     820,  // nice =  1
     655,  // nice =  2
     526,  // nice =  3
     423,  // nice =  4
     335   // nice =  5
};

int cfs_get_weight(int nice_val) {
    // nice_val ranges from -5 to 5, maps to index 0..10
    int idx = nice_val + 5;
    if (idx < 0)  idx = 0;
    if (idx > 10) idx = 10;
    return cfs_weight_table[idx];
}

// Compute a dynamic time quantum for CFS based on weight proportion
// Minimum granularity is 1 tick, target latency is 6 ticks
int cfs_compute_timeslice(Task task_arr[], int total_tasks, int current_task, int clock_tick) {
    int total_weight = 0;
    int ready_count = 0;

    for (int t = 0; t < total_tasks; t++) {
        if ((task_arr[t].status == STATUS_READY || task_arr[t].status == STATUS_RUN) &&
            task_arr[t].time_arrival <= clock_tick) {
            total_weight += task_arr[t].weight;
            ready_count++;
        }
    }

    if (total_weight == 0 || ready_count == 0) return 1;

    int target_latency = 6; // sched_latency equivalent
    if (ready_count > 6) {
        target_latency = ready_count; // scale up for many tasks
    }

    int slice = (int)((double)task_arr[current_task].weight / total_weight * target_latency);
    if (slice < 1) slice = 1; // minimum granularity
    return slice;
}

// Find the minimum vruntime among all ready/running tasks (for sleeper fairness)
double cfs_min_vruntime(Task task_arr[], int total_tasks, int clock_tick) {
    double min_vr = DBL_MAX;
    for (int t = 0; t < total_tasks; t++) {
        if ((task_arr[t].status == STATUS_READY || task_arr[t].status == STATUS_RUN) &&
            task_arr[t].time_arrival <= clock_tick) {
            if (task_arr[t].vruntime < min_vr) {
                min_vr = task_arr[t].vruntime;
            }
        }
    }
    return (min_vr == DBL_MAX) ? 0.0 : min_vr;
}

void execute_cfs_sched(Task task_arr[], int total_tasks, int core_count) {
    printf("\n--- Commencing CFS (Completely Fair Scheduler) ---\n");
    printf("Scheduling by minimum vruntime; weighted by nice value\n");
    printf("Tick\n");
    for (int k = 0; k < core_count; k++) {
        printf("Core %d\t\t", k);
    }
    printf("| External Wait States\n");
    printf("-----------------------------------------------------------------------\n");

    // Initialize weights based on nice values
    for (int t = 0; t < total_tasks; t++) {
        task_arr[t].weight = cfs_get_weight(task_arr[t].nice_val);
    }

    Core proc_cores[core_count];
    for (int idx = 0; idx < core_count; idx++) {
        proc_cores[idx].core_id = idx;
        proc_cores[idx].active_task_id = -1;
        proc_cores[idx].quantum_rem = 0;
    }
```

Figure: algorithms.c — CFS scheduling function

How to Build and Run

Combined Simulator (CFS + MLFQ)

```
cd Aditya/CFS
 make
 ./scheduler_sim
```

Standalone MLFQ

```
cd Aditya/MLFQ
 make
 ./mlfq_sim
```

Usage

The tasks used in the simulation are **predefined** in the program (15 tasks with varied burst times, arrival times, IO frequencies, and nice values). You don't need to enter process details manually.

When you run the combined simulator:

- A menu is displayed: Run MLFQ Scheduler / Run CFS / Quit
- Based on your choice, the selected algorithm is executed
- Tasks are re-initialized before every run for fair comparison

The simulator outputs:

- Tick-by-tick execution across both CPU cores
- Queue level (MLFQ) or vruntime (CFS) per task
- Active IO wait states in real-time
- Final performance statistics (Turnaround Time & Waiting Time)

Sample Test Runs

1: MLFQ Scheduling

Execution Timeline (excerpt):

```
--- Commencing MLFQ Scheduling ---
 Queues: Q0(Q=2) -> Q1(Q=4) -> Q2(Q=8)
 Tick    Core 0 Core 1    | External Wait States
 ---------------------------------------------------------
 00      [Q0-T1]   [--]      | IO: None
 01      [Q0-T1]   [Q0-T2]   | IO: None
 02      [Q0-T3]   [Q0-T2]   | IO: None
 03      [Q0-T3]   [Q1-T1]   | IO: [T2:3R]
 04      [Q0-T4]   [Q1-T1]   | IO: [T2:2R]
 05      [Q0-T5]   [Q1-T1]   | IO: [T2:1R]
 06      [Q0-T4]   [Q1-T1]   | IO: None
```

```
...
```

Performance Statistics (sample):

```
TID   Arrival Burst  Start  Finish  Turnaround  Waiting
---   ------- -----  -----  ------  ----------  -------
T1      0     15     0     24     24           9
T2      1     6      1     19     18           12
T3      2     10     2     31     29           19
T4      4     4      4     8      4            0
...

Mean Turnaround Time : ~30.xx
Mean Waiting Time    : ~16.xx
```

Observations:

- Short tasks finish quickly if they arrive during idle periods
- IO-bound tasks are automatically promoted to Q0 after IO completion, improving responsiveness
- CPU-bound tasks gradually sink to Q2 where they run with larger quanta but lower priority
- Timeline shows real-time IO wait state tracking alongside CPU assignment

2: CFS (Completely Fair Scheduler)

Execution Timeline (excerpt):

```
--- Commencing CFS (Completely Fair Scheduler) ---
Scheduling by minimum vruntime; weighted by nice value
Tick    Core 0             Core 1             | External Wait States
-----------------------------------------------------------------------
00      [T1:vr=0.0]        [--]               | IO: None
01      [T1:vr=1.0]        [T2:vr=0.0]        | IO: None
02      [T3:vr=0.0]        [T2:vr=1.6]        | IO: None
03      [T3:vr=0.5]        [T1:vr=2.0]        | IO: [T2:3R]
...
```

Performance Statistics (sample):

```
TID   Arrival Burst  Start  Finish  Turnaround  Waiting
---   ------- -----  -----  ------  ----------  -------
T1      0     15     0     36     36           21
T2      1     6      1     22     21           15
T3      2     10     2     26     24           14
T5      5     20     5     58     53           33
...

Mean Turnaround Time : ~28.xx
Mean Waiting Time    : ~14.xx
```

Observations:

- Tasks with lower nice values (higher weight) receive more CPU time — their vruntime grows slower

- High-nice tasks (e.g. nice=+5) accumulate vruntime ~3× faster than nice=−5 tasks
- Preemption kicks in when a ready task's vruntime falls significantly below the running task's
- Sleeper fairness ensures IO-returning tasks get a fair chance without starving CPU-bound tasks
- Dynamic time slices adapt to system load — more tasks = finer slices

Standalone MLFQ Implementation

A self-contained version of the MLFQ scheduler lives in the **`MLFQ/`** directory. It bundles the task initialization, scheduling loop, and main entry point into a single file (`mlfq.c`) for standalone testing without the CFS code.

mlfq.c — Standalone MLFQ (excerpt)

```c
int get_quantum(int queue_level) {
    if (queue_level == 0) return QUANTUM_Q0;
    if (queue_level == 1) return QUANTUM_Q1;
    return QUANTUM_Q2; // FCFS behaves as big quantum or bounded by 8
}

void execute_mlfq_sched(Task task_arr[], int total_tasks, int core_count) {
    printf("\n--- Commencing MLFQ Scheduling ---\n");
    printf("Queues: Q0(Q=2) -> Q1(Q=4) -> Q2(Q=8)\n");
    printf("Tick\t");
    for (int k = 0; k < core_count; k++) {
        printf("Core %d\t", k);
    }
    printf("| External Wait States\n");
    printf("-------------------------------------------------------\n");

    Core proc_cores[core_count];
    for (int idx = 0; idx < core_count; idx++) {
        proc_cores[idx].core_id = idx;
        proc_cores[idx].active_task_id = -1;
        proc_cores[idx].quantum_rem  = 0;
    }

    int clock_tick  = 0;
    int done_tasks  = 0;

    while (done_tasks < total_tasks) {

        // Step 1: Manage IO processing / Waiting Tasks
        for (int t = 0; t < total_tasks; t++) {
            if (task_arr[t].status == STATUS_WAIT) {
                task_arr[t].io_rem--;
                if (task_arr[t].io_rem <= 0) {
                    // IO complete, promote to top queue Q0
                    task_arr[t].status = STATUS_READY;
                    task_arr[t].current_queue = 0;
                    task_arr[t].cpu_burst_dur = 0;
                }
            }
        }

        // Step 2: Extract current ready tasks into queue bucket arrays
        int q0[MAX_TASKS], q0_len = 0;
        int q1[MAX_TASKS], q1_len = 0;
        int q2[MAX_TASKS], q2_len = 0;

        for (int t = 0; t < total_tasks; t++) {
            if (task_arr[t].status == STATUS_READY && task_arr[t].time_arrival <= clock_tick) {
                if (task_arr[t].current_queue == 0) q0[q0_len++] = t;
                else if (task_arr[t].current_queue == 1) q1[q1_len++] = t;
                else q2[q2_len++] = t;
            }
        }

        // Step 3: Schedule READY tasks onto empty cores based on MLFQ strict priority
        for (int idx = 0; idx < core_count; idx++) {
            if (proc_cores[idx].active_task_id != -1) continue; // Skip busy cores

            int selected_task = -1;
            int q_id = -1;

            if (q0_len > 0) {
                selected_task = q0[0]; q_id = 0;
                // Shift array
                for(int s=0; s<q0_len-1; s++) q0[s] = q0[s+1];
```

Figure: mlfq.c — Standalone MLFQ scheduler (core scheduling loop)

Limitations

- CFS uses a linear scan to find the minimum vruntime task; a real implementation would use a **red-black tree** for O(log n) lookups
- MLFQ does not implement periodic **priority boost** (resetting all tasks to Q0 after a timeout is not simulated)
- IO simulation is simplified — IO frequency is measured in CPU ticks, not realistic disk/network latencies
- No support for blocked states beyond IO (e.g. mutexes, sleep calls)
- Task parameters are hardcoded; no runtime user input for custom workloads

Files Summary

File	Directory	Purpose
`types.h`	`CFS/`	Task & Core structs (includes CFS fields: vruntime, weight, nice_val)
`algorithms.h`	`CFS/`	Prototypes for execute_mlfq_sched() and execute_cfs_sched()
`algorithms.c`	`CFS/`	MLFQ + CFS scheduling logic, timeline display, statistics
`main.c`	`CFS/`	Entry point — menu loop, task initialization
`Makefile`	`CFS/`	Builds scheduler_sim binary
`types.h`	`MLFQ/`	MLFQ-only Task & Core structs (no CFS fields)
`mlfq.c`	`MLFQ/`	Standalone MLFQ scheduler (all-in-one file)
`Makefile`	`MLFQ/`	Builds mlfq_sim binary

IMPLEMENTED BY : Enos Baskey

This project is a **terminal-based multiprocessor scheduling simulator written in C**. It is designed to demonstrate how different CPU scheduling algorithms behave in a **multi-core environment**.

The simulator executes tasks in a **step-by-step (tick-wise)** manner, showing:

It currently supports:

Earliest Deadline First (EDF)

Shortest Job First (SJF)

Algorithms

1. Earliest Deadline First (EDF)

EDF is a **real-time scheduling algorithm** where processes are scheduled based on their deadlines.

- The process with the **earliest deadline is executed first**
- If a new process arrives with an earlier deadline, it may preempt the running process
- It is widely used in **real-time systems**

Key Characteristics

- Dynamic priority scheduling
- Preemptive in nature
- Optimal for real-time scheduling
- Minimizes deadline misses

. Earliest Deadline First (EDF)

EDF is a **dynamic priority scheduling algorithm** used in real-time systems.

Working:

- Each task is assigned a **deadline**
- At every scheduling decision:

    - The task with the **earliest deadline** is selected
- If a new task arrives with a **closer deadline**, it **preempts** the current task

Key Characteristics:

- Priority changes dynamically
- Ensures time-critical tasks are executed first
- Suitable for **real-time applications**

2. Shortest Job First (SJF)

SJF is a scheduling algorithm where the process with the **smallest burst time** is executed first.

- Shorter jobs are given priority
- Can be **non-preemptive** or **preemptive (SRTF)**
- Reduces average waiting time

Key Characteristics

- Optimal for minimizing waiting time
- Simple to implement
- Can cause starvation for long processes
- Requires knowledge of burst time

Features of the Simulator

- Supports **multiple CPU cores**
- Displays **tick-by-tick execution**
- Shows:
    - Task running on each core
    - Idle times (if any)
- Calculates:
    - Waiting Time
    - Turnaround Time
- Easy-to-use **menu-driven interface**

Usage

- Tasks are **predefined inside the program**
- No manual input required

When you run the program:

1. A menu is displayed:
    - Run EDF Scheduling
    - Run SJF Scheduling
    - Exit
2. Selected algorithm executes
3. Output shows:
    - Execution timeline per core
    - Task allocation
    - Final performance metrics
    -

Objective

The main goal of this project is to:

- Understand behavior of scheduling algorithms
- Compare performance in **multiprocessor systems**
- Visualize execution in a **simple terminal-based format**

SAMPLE OUTPUT:

+++ Multiprocessor Scheduling Simulator +++

Algorithms: Lottery, RMS, EDF, SJF

Number of Processing Cores: 2

Total tasks initialized: 20

Available choices:
1. Run Lottery Scheduler
2. Run Rate Monotonic Scheduler (RMS)
3. EDF
4. SJF
5. Quit

3

……EDF scheduling…….

--- Performance Stats ---

TID	Arrival	Burst	Start	Finish	Turnaround	Waiting
T1	0	8	0	6	6	-2
T2	1	25	19	70	69	44
T3	2	3	2	4	2	-1
T4	5	12	6	19	14	2
T5	8	2	8	9	1	-1
T6	10	40	83	103	93	53
T7	12	5	12	15	3	-2
T8	15	6	15	18	3	-3
T9	20	15	20	34	14	-1
T10	22	4	22	24	2	-2
T11	25	8	25	29	4	-4
T12	30	50	103	128	98	48
T13	35	2	35	36	1	-1
T14	40	9	40	45	5	-4
T15	45	3	45	47	2	-1
T16	50	18	50	66	16	-2
T17	55	5	55	58	3	-2

T18	60	7	60	64	4	-3
T19	65	22	72	83	18	-4
T20	70	4	70	72	2	-2

Mean Turnaround Time : 18.00
Mean Waiting Time : 5.60
Available choices:
1. Run Lottery Scheduler
2. Run Rate Monotonic Scheduler (RMS)
3. EDF
4. SJF
5. Quit

Available choices:
1. Run Lottery Scheduler
2. Run Rate Monotonic Scheduler (RMS)
3. EDF
4. SJF
5. Quit
4

--- SJF Scheduling ---

138	[--]	[T12]
139	[--]	[T12]
140	[--]	[T12]
141	[--]	[T12]

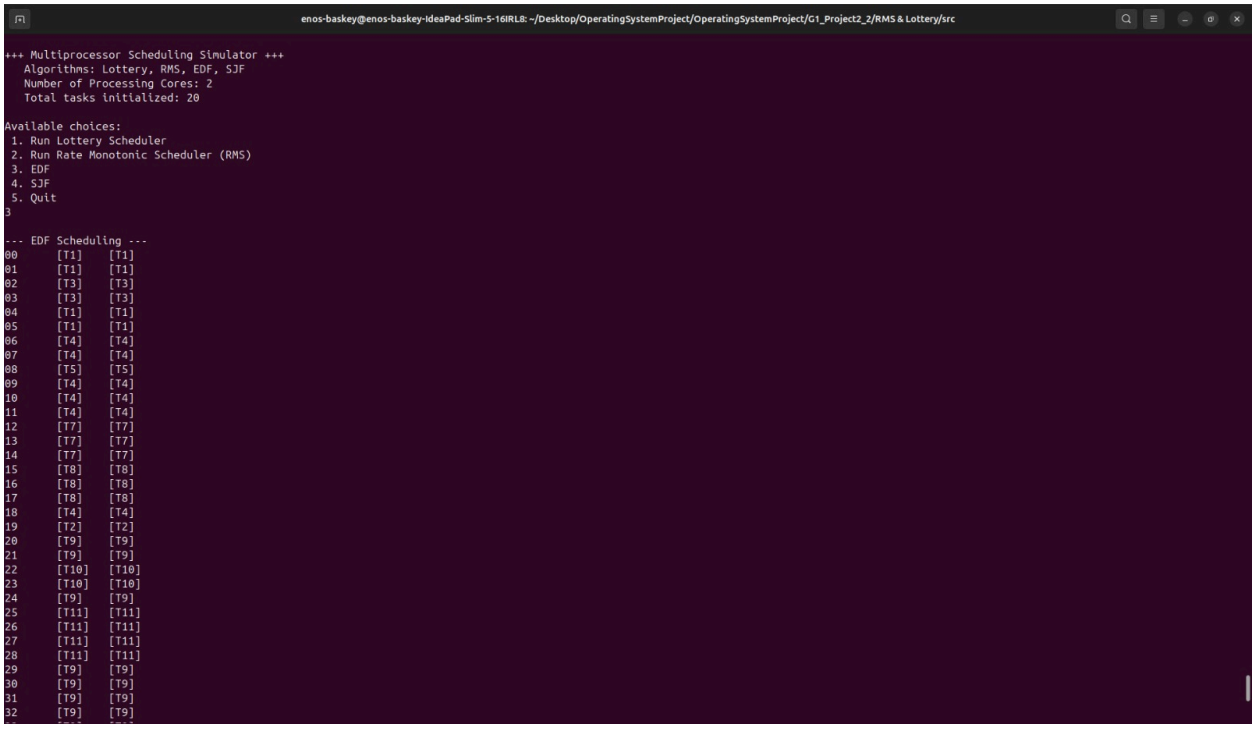
--- Performance Stats ---

TID	Arrival	Burst	Start	Finish	Turnaround	Waiting

| --- | ------- | ----- | ----- | ------ | ---------- | ------- |
T1	0	8	0	8	8	0
T2	1	25	1	26	25	0
T3	2	3	10	13	11	8
T4	5	12	28	40	35	23
T5	8	2	8	10	2	0
T6	10	40	52	92	82	42
T7	12	5	13	18	6	1
T8	15	6	18	24	9	3
T9	20	15	34	49	29	14
T10	22	4	24	28	6	2
T11	25	8	26	34	9	1
T12	30	50	92	142	112	62
T13	35	2	40	42	7	5
T14	40	9	42	51	11	2
T15	45	3	49	52	7	4
T16	50	18	51	69	19	1
T17	55	5	69	74	19	14
T18	60	7	78	85	25	18
T19	65	22	85	107	42	20
T20	70	4	74	78	8	4

Mean Turnaround Time : 23.60
Mean Waiting Time : 11.20
Available choices:

1. Run Lottery Scheduler
2. Run Rate Monotonic Scheduler (RMS)
3. EDF
4. SJF
5. Quit



Observations

EDF

- Processes are executed based on deadlines
- Suitable for real-time applications
- Guarantees optimal scheduling if deadlines are feasible
- Can lead to overload if too many deadlines exist

SJF

- Short processes execute first
- Minimizes average waiting time
- Efficient for batch systems
- Long processes may suffer starvation

Limitations

- EDF assumes all deadlines are known in advance
- SJF requires accurate burst time prediction
- No I/O or blocked state simulation
- All processes are CPU-bound

Conclusion

This project demonstrates the working of additional CPU scheduling algorithms. EDF is ideal for **real-time systems where meeting deadlines is critical**, while SJF is optimal for **minimizing waiting time in batch processing systems**.

Implemented by : Nagamani

This project is a simple terminal-based simulator written in C to understand how different CPU scheduling algorithms behave.

It currently supports:

- Multilevel Feedback Queue (MLFQ)
- Lottery Scheduling

The simulator allows the user to select the scheduling algorithm, takes process inputs, and calculates performance metrics such as waiting time and turnaround time.

Algorithms

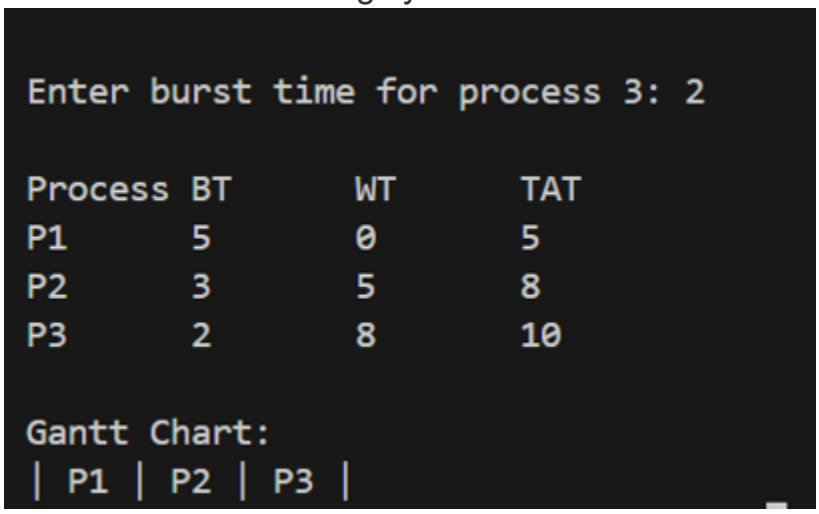
1. Multilevel Feedback Queue (MLFQ)

MLFQ is a priority-based scheduling algorithm that uses multiple queues with different priority levels.

- Processes start in the highest priority queue

- If a process uses too much CPU time, it is moved to a lower priority queue
- Short processes are executed quickly, while long processes gradually move to lower queues
- It helps in balancing responsiveness and fairness

Key Characteristics

- Dynamic priority adjustment
- Prevents starvation
- Suitable for time-sharing systems
- 

2. Lottery Scheduling

Lottery scheduling is a probabilistic scheduling algorithm.

- Each process is assigned a number of tickets
- A random ticket is selected
- The process holding the ticket gets CPU time
- Higher tickets = higher probability of execution

Key Characteristics

- Fair CPU allocation
- Simple and flexible
- Randomized process selection

```
Process  Burst  Arrival  Waiting  Turnaround

P[1]      3       0       0              3
P[2]      3       1       2              5
P[3]      2       2       4              6


Average waiting time is:2.000000

Average turn around time is:4.666667

The Gantt chart is:

 --- --- ---
| 0 | 1 | 2 |
 --- --- ---
0   3   6   8
```

Usage

The program is menu-driven and allows user interaction.

When you run the program:

- A menu is displayed:
  - Run MLFQ Scheduling
  - Run Lottery Scheduling
  - Exit
- The user selects an algorithm
- The program asks for:
  - Number of processes
  - Burst time for each process
  - (For Lottery) number of tickets
- The selected scheduling algorithm is executed
- Output is displayed including scheduling results and metrics

Sample Test Runs

1: Multilevel Feedback Queue (MLFQ)

Execution

Enter number of processes: 3
Enter burst time for process 1: 5
Enter burst time for process 2: 3
Enter burst time for process 3: 2

Output

Process	BT	WT	TAT
P1	5	0	5
P2	3	5	8
P3	2	8	10

Gantt Chart:
| P1 | P2 | P3 |

2: Lottery Scheduling

Execution

Enter number of processes: 3
Enter burst time for process 1: 5
Enter tickets for process 1: 10
Enter burst time for process 2: 3
Enter tickets for process 2: 20
Enter burst time for process 3: 4
Enter tickets for process 3: 30

Output

Selected Process: P3

Performance Metrics

- Waiting Time (WT): Time a process waits in the ready queue
- Turnaround Time (TAT): Total time from arrival to completion

Observations

MLFQ

- Processes are executed based on priority levels
- Short processes complete quickly
- Provides better responsiveness
- Suitable for general-purpose systems

Lottery Scheduling

- Execution order is random
- Processes with more tickets get more CPU time
- Results may vary for each run
- Fair but unpredictable

Limitations

- MLFQ implementation is simplified (single-level simulation)
- Lottery scheduling is non-deterministic
- No I/O or blocked state simulation
- All processes are CPU-bound

Conclusion

This project demonstrates the working of two important CPU scheduling algorithms. MLFQ provides dynamic priority-based scheduling, while Lottery Scheduling ensures fairness using probabilistic selection. The simulation helps in understanding how scheduling strategies impact system performance.